Equipment Management System — Security Implementation Details

For: Cyber Architecture Review (CAR) **Date:** 2026-03-28

---

1. Authentication Architecture

1.1 Azure AD Integration (Production)

Component	Configuration
Protocol	OAuth 2.0 / OpenID Connect
Provider	Microsoft Azure AD (Entra ID)
Tenant	UL Solutions Azure tenant
Scopes	`openid profile email offline_access User.Read`
Strategy	`passport-azure-ad` BearerStrategy
Metadata	`https://login.microsoftonline.com/{tenantID}/v2.0/.well-known/openid-configuration`
Issuer validation	Enabled in production, disabled in test mode
PII logging	Disabled (`loggingNoPII: true`)

1.2 Token Architecture

```
┌──────────────────────────────────────────────────────────┐
│                   Token Lifecycle                         │
│                                                           │
│  ┌──────────────┐   15 min    ┌──────────────────┐       │
│  │ Access Token ├──────────────►│ Expired          │       │
│  │ (HS256)      │              │ → Auto-refresh   │       │
│  └──────────────┘              │   60s before exp  │       │
│                                └──────────────────┘       │
│                                                           │
│  ┌──────────────┐   7 days    ┌──────────────────┐       │
│  │ Refresh Token├──────────────►│ Expired          │       │
│  │ (HS256)      │              │ → Re-login       │       │
│  │ type: refresh│              └──────────────────┘       │
│  └──────────────┘                                         │
│                                                           │
│  ┌──────────────┐   30 days   ┌──────────────────┐       │
│  │ Session      ├──────────────►│ Absolute expiry  │       │
│  │ (Absolute)   │              │ → Force re-login │       │
│  └──────────────┘              └──────────────────┘       │
│                                                           │
```

```
│  ┌──────────────┐   30 min   ┌──────────────────┐      │
│  │ Idle Timeout │──────────→│ Client-side      │      │
│  │ (Client)     │            │ → Sign out       │      │
│  └──────────────┘            └──────────────────┘      │
└────────────────────────────────────────────────────────┘
```

Parameter	Value	Source (SSOT)
Access token TTL	900s (15 min)	`shared-constants/auth-token.ts`
Refresh token TTL	604,800s (7 days)	`shared-constants/auth-token.ts`
Absolute session max	2,592,000s (30 days)	`shared-constants/auth-token.ts`
Refresh buffer	60s before expiry	`shared-constants/auth-token.ts`
Idle timeout	1,800s (30 min)	`shared-constants/auth-token.ts`
Signing algorithm	HS256 (HMAC-SHA256)	JWT strategy
Minimum secret length	32 chars (production)	env.validation.ts

1.3 Token Blacklist (Logout)

On logout, both access and refresh tokens are added to a blacklist with their remaining TTL:

Environment	Storage	Persistence
Production	Redis (ioredis)	Survives server restart
Development	In-memory Map	Lost on restart

The JWT strategy checks the blacklist on every request before validating the token.

1.4 Brute-Force Protection

Parameter	Value
Max login attempts	5 per email
Attempt window	15 minutes
Lock duration	15 minutes
Rate limit (Nginx)	10 req/min for auth endpoints
Rate limit (NestJS)	5 req/min for login endpoint

1.5 Multi-Tab Session Sync

Sessions are synchronized across browser tabs via BroadcastChannel:

- `LOGOUT` — Sign out all tabs
- `IDLE_LOGOUT` — Idle timeout propagation
- `ACTIVITY_RESET` — Reset idle timer across tabs

---

2. Authorization (RBAC)

2.1 Guard Chain

```
Request
  │
  ├─ @Public() ──────────────────────────► Skip all guards
  │
  ├─ JwtAuthGuard ───────────────────────► Validate JWT + blacklist check
  │      │
  │      ├─ @SkipPermissions() ──────────► Auth only, no RBAC
  │      │
  │      └─ PermissionsGuard ────────────► Check @RequirePermissions()
  │            │
  │            └─ Default-Deny mode ─────► Deny if no decorator found
  │
  └─ InternalApiKeyGuard ──────────────► X-Internal-Api-Key validation
         │                                (timing-safe comparison)
         └─ Dual key support ──────────► Current + Previous key rotation
```

2.2 Permission System

- **77 granular permissions** defined in `@equipment-management/shared-constants`
- **4 roles** with predefined permission sets
- **Default-Deny:** If no permission decorator is found, access is denied
- **CI enforcement:** `scripts/check-security-decorators.ts` validates all controller endpoints have security decorators

2.3 Server-Side User Extraction

```
// CORRECT — User ID from JWT token
@Patch(':uuid/approve')
async approve(@Param('uuid') uuid: string, @Request() req: AuthenticatedRequest) {
  const approverId = req.user?.userId; // Server-extracted
}

// NEVER — User ID from client body (spoofable)
async approve(@Body() dto: { approverId: string }) { ... }
```

---

3. Input Validation

3.1 Zod Pipeline

All API endpoints use Zod schema validation (NOT class-validator):

```
Client Request
  │
  └─ ZodValidationPipe ──► Parse with Zod schema
       │                      ├─ Valid → Continue to controller
       │                      └─ Invalid → 400 Bad Request
```

```
      │                           {
      │                             code: 'VALIDATION_ERROR',
      │                             message: 'Validation failed',
      │                             errors: [{ path, message }]
      │                           }
      │
      └─ CI check: pnpm security:check
           └─ Warns if @Body() used without validation pipe
```

3.2 File Upload Validation

- **Magic byte verification:** File type checked by content, not extension
- **Allowed types:** PDF, JPEG, PNG, GIF, DOCX, XLSX
- **Max size:** 100MB (Nginx limit)
- **Path traversal prevention:** All file paths validated against base directory

---

4. Data Protection

4.1 SQL Injection Prevention

- **Drizzle ORM:** All queries use parameterized statements
- **No raw SQL:** Application code does not execute raw SQL strings
- **ESLint rules:** `no-restricted-imports` prevents direct database driver imports

4.2 XSS Prevention

Layer	Protection
Helmet CSP	`script-src: 'self'` only (production)
React 19	Automatic output escaping
X-Content-Type-Options	`nosniff`
X-XSS-Protection	Enabled

4.3 CSRF Prevention

- JWT Bearer token authentication (not cookies for API auth)
- CORS origin whitelisting (`FRONTEND_URL` only)
- SameSite cookie attributes for session cookies

4.4 Clickjacking Prevention

- `X-Frame-Options: DENY` (Helmet)
- `Permissions-Policy: camera=(), microphone=(), geolocation=()`

---

5. Encryption

5.1 Transport Encryption (TLS)

Setting	Configuration
Protocols	TLS 1.2, TLS 1.3

Cipher preference	Server
Cipher suites	ECDHE-ECDSA-AES128-GCM-SHA256, ECDHE-RSA-AES128-GCM-SHA256, ECDHE-ECDSA-AES256-GCM-SHA384, ECDHE-RSA-AES256-GCM-SHA384, ECDHE-ECDSA-CHACHA20-POLY1305, ECDHE-RSA-CHACHA20-POLY1305, DHE-RSA-AES128-GCM-SHA256, DHE-RSA-AES256-GCM-SHA384
HSTS	max-age=63072000; includeSubDomains; preload
OCSP stapling	Enabled
Session tickets	Disabled (forward secrecy)
SSL session cache	Shared 10MB, 1-day timeout

All cipher suites are **NIST SP 800-175B approved**.

5.2 JWT Signing

Parameter	Value
Algorithm	HS256 (HMAC-SHA256)
Access token secret	JWT_SECRET (384-bit entropy, openssl rand)
Refresh token secret	REFRESH_TOKEN_SECRET (384-bit, must differ from JWT_SECRET)
Key rotation	Supported via INTERNAL_API_KEY_PREVIOUS

5.3 Database Connection Encryption

- **DB_SSL=true:** Enables SSL for PostgreSQL connections
- **DB_SSL_REJECT_UNAUTHORIZED:** Certificate validation control (default: true)
- Docker internal network deployment: SSL optional (containers share isolated network)

5.4 Redis Connection Encryption

- **REDIS_TLS=true:** Enables TLS for Redis connections
- **REDIS_URL=rediss://:** URL-scheme TLS support
- Docker internal network deployment: TLS optional

---

6. Audit & Logging

6.1 Audit Trail

All state-changing operations are logged via the `@AuditLog()` decorator:

```
@AuditLog({
  action: 'approve',
  entityType: 'calibration',
```

```
  entityIdPath: 'params.uuid',
})
@Patch(':uuid/approve')
approve() { ... }
```

Audit events are processed asynchronously (non-blocking) and include:

- User ID, email
- Action performed
- Entity type and ID
- Timestamp
- Request details

6.2 Authentication Events

Event	Trigger	Data
`audit.auth.success`	Successful login	userId, email, method (local/azure_ad)
`audit.auth.failed`	Failed login	email, reason (invalid_credentials/account_locked)

6.3 Application Logging

Component	Tool	Configuration
Application	Winston	Daily rotation, structured JSON
Container	Promtail → Loki	14-day retention
Metrics	Prometheus → Grafana	15-day retention, 15s scrape
Alerts	Alertmanager → Slack	Critical/Warning channels

---

7. Rate Limiting

7.1 Nginx Layer

Zone	Rate	Burst	Scope
global	100 req/min	200	All requests
api	60 req/min	30	`/api/*` endpoints
auth	10 req/min	5	`/api/auth/(login|signin|token|refresh)`

7.2 NestJS Layer

Preset	Limit	Window	Applied To
LOGIN	5 req	60s	POST /api/auth/login
TOKEN_REFRESH	10 req	60s	POST /api/auth/refresh
Short	20 req	1s	Default

Medium	100 req	10s	Configurable
Long	300 req	60s	Configurable

---

8. Concurrency Control (CAS)

All state-changing operations use Compare-and-Swap (optimistic locking):

```
Client sends: { version: 3, status: 'approved' }
  │
  └─ UPDATE equipment
     SET status = 'approved', version = version + 1
     WHERE id = ? AND version = 3
       │
       ├─ 1 row affected → Success (version now 4)
       │
       └─ 0 rows affected → Conflict
            ├─ Entity not found → 404
            └─ Version mismatch → 409 VERSION_CONFLICT
                 │
                 └─ Cache invalidation (prevent stale retry)
```

CAS-protected entities: equipment, checkouts, calibrations, non-conformances, disposal_requests, equipment_imports, equipment_requests, software_history

---

9. Dependency Security

Tool	Purpose	Integration
Dependabot	Automated dependency updates	Weekly PR creation (npm + GitHub Actions)
gitleaks	Secret scanning	CI Gate 1 (every push/PR)
npm audit	Vulnerability detection	CI Gate 3 (High/Critical fails build)
pnpm overrides	Known vulnerability mitigation	9 packages currently managed
pnpm verify-store-integrity	Package tampering detection	Enabled in .npmrc
Husky + lint-staged	Pre-commit validation	Every commit

---

10. Security Headers

Applied via Helmet middleware and Nginx:

Header	Value	Source
Strict-Transport-Security	max-age=63072000; includeSubDomains; preload	Nginx

X-Frame-Options	DENY	Helmet
X-Content-Type-Options	nosniff	Nginx + Helmet
Referrer-Policy	strict-origin-when-cross-origin	Nginx + Helmet
Permissions-Policy	camera=(), microphone=(), geolocation=()	Nginx
Content-Security-Policy	default-src 'self'; script-src 'self'	Helmet
X-XSS-Protection	1; mode=block	Helmet
X-Powered-By	(removed)	Helmet

---

11. Error Handling

11.1 Global Exception Filter

Processing order: `AppError` → `ZodError` → `HttpException` → `unknown`

All error responses include a machine-readable `code` field:

```
{
  "code": "VERSION_CONFLICT",
  "message": "Resource was modified by another user",
  "timestamp": "2025-07-14T10:00:00.000Z",
  "currentVersion": 4,
  "expectedVersion": 3
}
```

11.2 Security-Related Error Codes

Code	HTTP Status	Meaning
AUTH_PRODUCTION_AZURE_ONLY	403	Local login blocked in production
AUTH_INVALID_CREDENTIALS	401	Wrong email/password
AUTH_ACCOUNT_LOCKED	401	Brute-force protection triggered
AUTH_AZURE_AD_FAILED	401	Azure AD validation failed
AUTH_TOKEN_BLACKLISTED	401	Token was invalidated (logout)
AUTH_SESSION_EXPIRED	401	30-day absolute max exceeded
AUTH_USER_INACTIVE	401	User account deactivated
VERSION_CONFLICT	409	CAS optimistic locking conflict